JAVA 연산자

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Introduction to
Java
C. Thomas Wu
C++

연산자란?

연산자와 피연산자

연산자란?

연산자의 종류

연산 형식	연산자의 종류	연산자	연산 방향
단항	증감	++, --, +, -, !, ~	←
이항	산술	+, -, *, /, %	→
	시프트	>>, <<, >>>	→
	비교	==, >, <, >=, <=, !=	→
	비트	&, |, ^, ~	→
	논리	&&, ||, !, ^	→
삼항	삼항	? :	→
이항	대입	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=	→

연산자의 연산 방향

연산자의 종류에 따라 연산의 방향이 다르게 정의되어 있음

단항 연산자와 대입 연산자는 오른쪽에서 왼쪽으로 연산이 이루어짐

예

```
int num1 = 10 ;
```

```
num1 = num2 = num3 ;
```

②　①

```
++ num ;
```

```
!check ;
```

이항 연산자와 삼항 연산자는 왼쪽 항에서 오른쪽 항의 방향으로 연산이 이루어짐

예

```
a + b
```

```
a > b
```

```
a && b
```

연산자의 우선순위

여러 연산자의 우선순위는 이미 우리가 알고 있는 것

$\boxed{-x} + y$

$x + \boxed{y * z}$

$\boxed{x + z} > \boxed{y + 10}$

$x = \boxed{y + 10}$

연산자의 우선순위

단항 > 산술 > 비교 > 논리 > 삼항 > 대입

우선순위	연산자의 종류	
1	.(점), [대괄호], (괄호)	우선순위 높음
2	++(postfix), --(postfix)	
3	+(양수), -(음수), ++(prefix), --(prefix), ~, !	
4	(casting type)	
5	*, /, %	
6	+(더하기), -(빼기)	
7	<<, >>, >>>	
8	>, <, <>, <=, instanceof	
9	==, !=	
10	&	
11	^	
12	|	
13	&&	
14	||	
15	(condition) ? (식) : (식)	
16	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=	우선순위 낮음

단항 연산자

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

부호 연산자

+, -

변수의 부호를 바꿔주는 연산자

연산자에는 +, - 두 개의 연산자가 있음

+ 연산자

실제 아무런 역할을 하지 않음

- 연산자

변수에 -1을 곱해 부호를 변경시킴

부호 연산자

+, -

```
package chapter03;

public class AdditiveOperator {

    public static void main(String[] args) {
        int num1 = 5;
        int num2 = +num1; // + 연산은 변수에 1을 곱하는 연산을 수행
        int num3 = -num1; // - 연산은 변수에 -1을 곱하는 연산을 수행
        System.out.println("num1 => " + num1);
        System.out.println("num2 => " + num2);
        System.out.println("num3 => " + num3);
    }
}
```

논리 부정 연산자

!

변수가 가지는 논리값을 반대의 논리값으로 변경해 주는 연산자

피연산자가 Boolean 타입일 때 사용

논리값이 true 이면 false로, false일 때는 true로 변경

논리 부정 연산자

!

```
package chapter03;

public class NotOperator {

    public static void main(String[] args) {
        int num1 = 5;
        int num2 = 13;
        boolean led = true;

        System.out.println("led 변수의 논리 값 => " + led);
        System.out.println("led 변수의 논리 값의 변경 => " + !led); // 논리값을 반대 값으로 변경
        System.out.println("num1>num2의 결과 : " + (num1 > num2));
        System.out.print("num1>num2의 결과 값을 반전 한 결과 : ");
        System.out.println(!(num1 > num2)); // 비교 연사의 결과 논리값을 반대 논리 값으로 변경
    }
}
```

증감연산자

전위형(prefix)

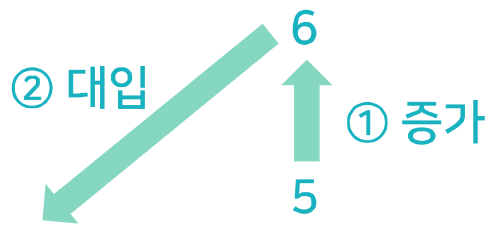
변수에 저장된 값(피연산자)을 1씩 증가시키거나 감소시키는 연산자

전위형 증감연산자는 연산 기호가 피연산자의 앞에 붙음

다른 연산자와 같이 쓰일 때 증감 연산을 우선 처리하고 다른 연산을 처리

예

```
int num1 = 5;
```




```
num2 = ++num1;
```

연산자	연산자의 기능	예	연산 방향
++	피연산자에 저장된 값에 1 증가함	++num	←
--	피연산자에 저장된 값을 1 감소함	--num	←

증감연산자

전위형(prefix)

```
package chapter03;

public class PrefixOperator {

    public static void main(String[] args) {
        int num1 = 5;
        int num2 = ++num1 ; // num1의 값을 먼저 1증가 시킨 후 대입 연산을 수행

        System.out.println("num1 의 값 : " + num1); // 6
        System.out.println("num2 의 값 : " + num2); // 6
        System.out.println("++num1 의 값 : " + ++num1); // 7,
        // num1의 값을 먼저 1증가시킨 후 문자열 연산이 수행
    }
}
```

증감연산자

후위형(postfix)

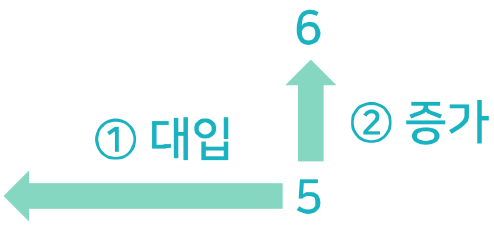
변수에 저장된 값(피연산자)을 1씩 증가시키거나 감소시키는 연산자

후위형 증감 연산자 연산은 기호가 피연산자의 뒤에 붙음

다른 연산자와 같이 사용할 때 다른 연산을 먼저 처리한 후 증감 연산을 처리

예

```
int num1 = 5;
```




```
num2 = num1++;
```

연산자	연산자의 기능	예	연산 방향
++	피연산자에 저장된 값에 1 증가함	num++	←
--	피연산자에 저장된 값을 1 감소함	num--	←

증감연산자

후위형(postfix)

```
package chapter03;

public class PostfixOperator {

	public static void main(String[] args) {
		int num1 = 5;
		int num2 = num1++ ; // 대입 연산 후 num1의 값을 1 증가시킴

		System.out.println("num1 의 값 : " + num1); // 6
		System.out.println("num2 의 값 : " + num2); // 5
		System.out.println("num1++ 의 값 : " + num1++);
		// 6 출력 후 num1의 값 1 증가
	}
}
```

증감연산자의 순서

```
package chapter03;

public class UnaryOperator {

	public static void main(String[] args) {
		int num1 = 30;

		System.out.println(num1++); // 30출력 ->(num1->31)
		System.out.println(++num1); // (num1->32) -> 32출력
		System.out.println(num1--); // 32출력 ->(num1->31)
		System.out.println(--num1); // (num1->30) -> 30출력
		System.out.println(num1++ + ++num1); // 30+((30+1)+1)=62
	}
}
```

형 변환 연산자

원래 가지고 있던 데이터 타입을 다른 종류의 데이터 타입으로 변환하는 연산자

피연산자 앞에 괄호()를 사용하고 괄호 안에 변경하고자 하는 데이터 타입을 명시해 줌

형 변환 연산자

```
package chapter03;

public class CastingOperator {

	public static void main(String[] args) {
		byte num1 = 10;
		byte num2 = 20;

		//byte num3 = num1 + num2;  // 오류 발생
		// num1 변수와 num2 변수가 int 타입으로 변경됨
		// 덧셈의 결과도 int 타입이 됨
		byte num3 = (byte)(num1 + num2); // 덧셈의 결과를 byte. 타입으로 변경
	}
}
```

이항 연산자

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

산술 연산자

이항연산자 중 가장 많이 사용되는 연산자

더하기, 빼기, 곱하기, 나누기, 나머지

연산자	연산자의 기능	예	연산 방향
+	왼쪽에 피연산자와 오른쪽 피연산자의 값을 더함	10 + 20	→
-	왼쪽에 피연산자와 오른쪽 피연산자의 값을 뺌	100 - 10	→
*	왼쪽에 피연산자와 오른쪽 피연산자의 값을 곱함	5 * 2	→
/	왼쪽에 피연산자에서 오른쪽 피연산자의 값으로 나눔	10 / 2	→
%	왼쪽에 피연산자에서 오른쪽 피연산자의 값으로 나눈 후, 얻는 나머지 값을 반환함	10 % 3	→

산술 연산자

```
package chapter03;

public class ArithmeticOperator {

	public static void main(String[] args) {
		int num1 = 30;
		int num2 = 2;
		float num3 = 2.0f;

		System.out.println(num1 + num2); // 30 + 2 = 32
		System.out.println(num1 - num2);  // 30 - 2 = 28
		System.out.println(num1 * num2);  // 30 * 2 = 60
		System.out.println(num1 / num2);  // 30 / 2 =15
		System.out.println(num1 / num3);  // 30 / 2.0f ➜ 30.0f / 2.0f = 15.0f
		System.out.println(num1 % num2); // 30 / 2.0f 의 나머지 는 0
	}
}
```

산술 연산자

정수형 연산 패턴

산술 연산의 정수형 연산은 int 타입으로 변경된 후 연산함

표현의 범위가 큰 자료형으로 자동형 변환이 되어 연산함

1. byte + byte → int + int → 결과 int
2. byte + int → int + int → 결과 int
3. long + int → long + long → 결과 long
4. float + int → float + float → 결과 float
5. float + double → double + double → 결과 double

산술 연산자

정수형 연산 패턴

```
package chapter03;

public class ArithmeticOperator2 {

    public static void main(String[] args) {
        float num1 = 10.f;
        float num2 = 3.0f;

        System.out.println(num1 / num2);
        System.out.println(num1 % num2);
    }
}
```

산술 연산자

문자형 연산 패턴

```
package chapter03;

public class CharTypeOperator {

    public static void main(String[] args) {
        char c1 = 'A'; // 유니코드 값이 저장
        int n1 = c1;

        // char c2 = c1 + 1;  // 오류가 발생합니다.

        int n2 = c1 + 1; // c1이 int 타입으로 변환되고 int+int 연산이 됩니다.

        System.out.println(c1);
        System.out.println(n1);
        System.out.println(n2);
    }
}
```

산술 연산자

문자형 증감 연산 패턴

```
package chapter03;

public class CharTypeOperator2 {

    public static void main(String[] args) {
        char c = 'A';

        System.out.println(c++); // char 타입의 증감 연산은 char 타입을 유지 A 출력
        System.out.println(c--); // 'B' 출력
        System.out.println(c); // 'A' 출력
        System.out.println(++c); // 'B' 출력
        System.out.println(--c); // 'A' 출력
    }
}
```

산술 연산자

문자열 간의 더하기 연산

문자열의 경우 모든 산술연산이 가능한 것이 아니라 + 연산만 가능

+ 연산자는 문자열을 붙여 하나의 문자열로 만들어주는 연산을 함

1. "문자열" + "문자열" → "문자열문자열"
2. "문자열" + true → "문자열" + "true" → "문자열true"
3. "문자열" + 100 → "문자열" + "100" → "문자열100"
4. "문자열" + 10.2f → "문자열" + "10.2" → "문자열10.2"
5. "문자열" + null → "문자열" + "null" → "문자열null"

관계 연산자

관계 연산자는 두 피연산자의 데이터를 비교할 때 사용함

비교 연산도 산술 연산과 같이 연산 시에 자료형 타입을 일치시켜 연산함

비교 연산 후 반환 값은 true 또는 false의 논리값을 반환함

프로그램의 흐름을 처리하는 if 문 또는 반복문의 반복의 조건을 정의할 때 사용함

관계 연산자

연산자	연산자의 기능	예	연산 방향
>	왼쪽에 피연산자가 오른쪽 피연산자보다 크면 true를 반환	1 > 0	→
<	왼쪽에 피연산자가 오른쪽 피연산자보다 작으면 true를 반환	0 < 1	→
>=	왼쪽에 피연산자가 오른쪽 피연산자보다 크거나 같으면 true를 반환	1 >= 0	→
<=	왼쪽에 피연산자가 오른쪽 피연산자보다 작거나 같으면 true를 반환	1 <= 0	→
==	왼쪽에 피연산자와 오른쪽 피연산자가 같으면 true를 반환	1 == 1	→
!=	왼쪽에 피연산자와 오른쪽 피연산자가 다르면 true를 반환	1 != 0	→

관계 연산자

```
package chapter03;

public class RelationalOperator {

    public static void main(String[] args) {
        int num1 = 30;
        int num2 = 2;

        System.out.println(num1 == num2); // false
        System.out.println(num1 > num2);   // true
        System.out.println(num1 < num2);   // false
        System.out.println(num1 >= num2); // true
        System.out.println(num1 <= num2); // false
        System.out.println(num1 != num2);  // true
    }
}
```

관계 연산자

참조형 변수의 비교

```
package chapter03;

public class RelationalOperator2 {

    public static void main(String[] args) {
        String str1 = "JAVA";
        String str2 = "Program";

        System.out.println(str1 == str2); // 참조 주소 값 == 참조 주소 값
        System.out.println(str1 != str2);  // 참조 주소 값 != 참조 주소 값

        // System.out.println(str1 > str2); //오류
    }
}
```

관계 연산자

비교 연산과 자동 형 변환

```
package chapter03;

public class RelationalOperator3 {

    public static void main(String[] args) {
        char c = 'A';
        int num = 10;

        System.out.println( c > 10 ); //  65 > 10
        System.out.println( '1' > '0'); // 49 > 48
        System.out.println( num == 10f ); // int == float -> float == float
    }
}
```

논리 연산자

피연산자의 논리값을 가지고 비교연산 후 참(true), 거짓(false) 값을 반환

논리곱(AND), 논리합(OR), 배타적 논리합(XOR), 논리부정(NOT)가 있음

연산자	연산자의 기능	연산 방향
&&	왼쪽에 피연산자와 오른쪽 피연산자의 논리 값이 모두 true이면 true를 반환하고, 이외에는 false를 반환	→
||	피연산자 중 하나만 true면 true를 반환	→
^	피연산자의 논리값이 서로 다를 경우만 true 반환	→
!	피연산자의 논리값을 바꿈	→

논리 연산자

논리표

a	b	a && b	a || b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

논리 연산자

```
package chapter03;

public class LogicalOperator {

    public static void main(String[] args) {
        int num1 = 30;
        int num2 = 2;

        System.out.println(num1 > 10 && num1 < 50); // true && true -> true
        System.out.println(num1 < 10 && num1 < 50); // false && true -> false
        System.out.println(num1 > 10 || num2 > 10); // true || false -> true
        System.out.println(num1 < 10 || num2 > 10); // false || false -> false
    }
}
```

비트 연산자

저장된 데이터를 비트 단위로 연산을 하는 연산자

비트 단위 연산자는 정수형 데이터에서만 사용 가능함

2진수나 16진수 데이터를 사용하여 연산함

비트 논리 연산자와 비트 시프트 연산자가 있음

비트 연산자

비트 논리 연산자

연산자	연산자의 기능
&	왼쪽에 피연산자와 오른쪽 피연산자의 비트의 AND 연산
|	왼쪽에 피연산자와 오른쪽 피연산자의 비트의 OR 연산
^	왼쪽에 피연산자와 오른쪽 피연산자의 비트의 XOR 연산
~	피연산자의 비트의 NOT 연산

비트 연산자

비트 논리 연산자

1 &(AND) 연산자

연산하는 두 비트가 모두 1일 때만 1을 반환하고 다른 경우는 모두 0을 반환

10진수	저장 비트							
5	0	0	0	0	0	1	0	1
	&	&	&	&	&	&	&	&
3	0	0	0	0	0	0	1	1
5&3	0	0	0	0	0	0	0	1

1 & 1 ➡ 1　　0 & 1 ➡ 0

1 & 0 ➡ 0　　0 & 0 ➡ 0

결과 ➡ 1

비트 연산자

비트 논리 연산자

2 |(OR) 연산자

연산하는 두 비트 중 하나라도 1이면 1을 반환하고 두 비트가 모두 0일 때만 0을 반환

10진수	저장 비트							
5	0	0	0	0	0	1	0	1
|	|	|	|	|	|	|	|	|
3	0	0	0	0	0	0	1	1
5|3	0	0	0	0	0	1	1	1

1 | 1 ➡ 1　　0 | 1 ➡ 1

1 | 0 ➡ 1　　0 | 0 ➡ 0

결과 ➡ 7

비트 연산자

비트 논리 연산자

3 ^(XOR) 연산자

연산하는 두 비트가 서로 다르면 1을 반환하고 서로 같으면 0을 반환

10진수	저장 비트							
5	0	0	0	0	0	1	0	1
	^	^	^	^	^	^	^	^
3	0	0	0	0	0	0	1	1
5^3	0	0	0	0	0	1	1	0

1 ^ 1 ➡ 0　　0 ^ 1 ➡ 1

1 ^ 0 ➡ 1　　0 ^ 0 ➡ 0

결과 ➡ 6

비트 연산자

비트 논리 연산자

4 ~(NOT) 연산자

‹···› 해당 비트가 1이면 0을 반환하고 0이면 1을 반환

비트 연산자

비트 논리 연산자

```
package chapter03;

public class BItOperator {

	public static void main(String[] args) {
		int num1 = 5;
		int num2 = 3;

		System.out.println(5 & 3); // 1
		System.out.println(5 | 3); // 7
		System.out.println(5 ^ 3); // 6
		System.out.println(~5); // -6
	}
}
```

시프트 연산자

정수형 데이터를 지정한 비트만큼 왼쪽 또는 오른쪽으로 이동시킴

왼쪽으로 한번 시프트 하는 경우 2를 곱한 결과와 같음

예 왼쪽으로 n 번 이동하는 경우 2의 n승 한 것과 같음

오른쪽으로 한번 시프트 하는 경우는 2로 나누는 것과 같음

예 오른쪽으로 n번 시프트 하는 경우는 2의 n승으로 나누는 것과 같음

연산자	연산자의 기능
A<<B	왼쪽에 피연산자 A를 B 비트만큼 왼쪽으로 이동시키고 오른쪽 비트는 0으로 채움
A>>B	왼쪽에 피연산자 A를 B 비트만큼 오른쪽으로 이동시키고 왼쪽 비트는 양수일 경우 0으로 채우고, 음수일 경우 1로 채움
A>>>B	왼쪽에 피연산자 A를 B 비트만큼 오른쪽으로 이동시키고 왼쪽 비트는 항상 0으로 채움

시프트 연산자

<<

정수 데이터를 왼쪽으로 1번 시프트 하는 연산

시프트 연산자

>>

정수 데이터를 오른쪽으로 1번 시프트 하는 연산

부호 비트 값에 따라서 공백 비트의 값이 부호 비트 값으로 채워 짐

부호 비트가 1이면 시프트 되면서 생기는 공백 비트에 1이 채워지고 0이었으면 0이 채워지게 됨

시프트 연산자

>>

10진수	저장 비트							
4	0	0	0	0	0	1	0	0
전체 비트를 오른쪽으로 시프트 →	0	0	0	0	0	1	0	0
	0	0	0	0	0	0	1	0
4>>1	0	0	0	0	1	0	1	0

범위를 벗어나는 비트는 삭제

비어 있는 비트는 부호 비트가 1인 경우 1이 채워지고, 0인 경우 0으로 채움

결과 → 2

시프트 연산자

>>>

정수 데이터 4의 비트를 오른쪽으로 1번 시프트 하고 이동된 데이터는 항상 0으로 채우는 연산이 됨

10진수	저장 비트
4	0 0 0 0 0 1 0 0
전체 비트를 오른쪽으로 시프트 →	0 0 0 0 0 1 0 0
	0 0 0 0 0 0 1 0
	범위를 벗어나는 비트는 삭제
비어 있는 부호 비트는 무조건 0으로 채움	
4>>>1	0 0 0 0 0 0 1 0
	결과 → 2

시프트 연산자

```
package chapter03;

public class BitOperator2 {

	public static void main(String[] args) {
		int num1 = 5;
		int num2 = 4;

		System.out.println(num1<<1);//10
		System.out.println(num1<<3);//40
		System.out.println(num2>>1);//2
		System.out.println(num2>>2);//1
		System.out.println(num2>>>1);//2
	}
}
```

대입 연산자

오른쪽 변의 값을 연산자 왼쪽에 있는 변수에 대입하는 연산자

모든 연산자 중 우선순위가 가장 낮으며, 연산식의 맨 마지막에 처리됨

대입 연산자도 연산이기 때문에 왼쪽 변에 있는 변수의 타입과 저장하고자 하는 변수 혹은 리터럴의 데이터 타입이 일치해야 함

서로 다른 타입의 연산에서는 자동으로 형 변환이 이루어짐

```
num = 10;
```

대입 연산자

②　①

```
num1 = num2 = num3
```

- 먼저 num2 변수에 변수 num3 가진 값을 대입 후 변수 num1에 변수 num2가 가지는 값을 저장하는 순서로 연산이 이루어짐

```
float number = 10;  →  float number = 10f;
```

- float 타입의 변수 number에 int 타입 리터럴 10을 저장하는 연산
- 리터럴 10이 자동으로 float 타입으로 형 변환 되어 대입 연산 실행

대입 연산자

복합 대입 연산자

대입 연산자와 다른 연산자를 묶어 다양한 형식의 연산자

대입 연산자 앞에 다른 연산자가 붙어 복합 연산자가 됨

숫자 기반의 산술 연산자 또는 비트 연산자와 대입 연산자를 묶어서 사용

문자열의 경우 +연산자와 대입 연산자를 사용하는 복합 대입 연산자를 사용

대입 연산자

복합 대입 연산자

일반 연산		복합 대입 연산
num = num + 1	→	num += 1
num = num - 1	→	num -= 1
num = num * 1	→	num *= 1
num = num / 1	→	num /= 1
num = num % 1	→	num %= 1
num = num & 1	→	num &= 1
num = num | 1	→	num |= 1
num = num ^ 1	→	num ^= 1
num = num << 1	→	num <<= 1
num = num >> 1	→	num >>= 1
num = num >>> 1	→	num >>>= 1

대입 연산자

```
package chapter03;

public class AssignmentOperator {

    public static void main(String[] args) {

        int num = 10;

        num += 5;// num=10+5
        System.out.println(num);
        num -= 5;// num=15-5
        System.out.println(num);
        num *= 2;// 10*2
        System.out.println(num);
        num /= 2;// 20/2
        System.out.println(num);
        num %= 3;// 20%2
        System.out.println(num);
```

대입 연산자

```
        short snum1 = 20;
        short snum2 = 30;
        snum1 += snum2; //
        // snum1 = snum1 + snum2;//오류 발생 -> snum1과 snum2는 int 타입으로 변환되어 연산
        System.out.println(snum1);

        num=snum1=snum2;
        System.out.println("num = "+num);
        System.out.println("snum1 = "+snum1);
        System.out.println("snum2 = "+snum2);
    }
}
```

삼항 연산자

조건문을 대체하는 연산자

세 개의 피연산자를 사용하는 연산자

조건에 의해 문장이 실행되기 때문에 간단한 조건문은 삼항 연산자로 대체

(조건식) ? 조건이 true일 때 반환 값 : 조건이 false일 때 반환 값 ;

삼항 연산자

```
package chapter03;
public class TernaryOperator {
    public static void main(String[] args) {
        int num1 = 2;
        int num2 = 5;
        int min = (num1 < num2) ? num1 : num2;
        System.out.println(min);
        int max = (num1 > num2) ? num1 : num2;
        System.out.println(max);
        String result = num1 % 2 == 0 ? "짝수" : "음수";
        System.out.println("num1은 " + result);
        result = num2 % 2 == 0 ? "짝수" : "음수";
        System.out.println("num2는 " + result);
    }
}
```